

Opcodes of the Hermes virtual machine

With the opcode byte value indicated respective to each stable version of the React Native Hermes VM bytecode format, as well as the auto-generated documentation from the "BytecodeList.def" file from the Hermes VM source tree.

This list of opcodes is used by the [hermes-dec](#) open-source reverse engineering project.

Instruction	Bytecode version to opcode	Documentation
-------------	----------------------------	---------------

51	58	59	60	62	71	74	76	83	84	89
----	----	----	----	----	----	----	----	----	----	----

Add	21	21	21	21	21	21	21	21	21	22	22	22	Reg8, Reg8, Reg8 (total size 3)
													Arg1 = Arg2 + Arg3 (JS addition/ concatenation)

													Reg8, Reg8 (total size 2)
AddEmptyString	113	115	116	116	116	116	116	116	121	121	126		Convert a value to a string as if evaluating the expression: Arg1 = "" + Arg2 In practice this means Arg1 = ToString(ToPrimitive(Arg2, PreferredType::NONE)) with ToPrimitive (ES5.1 9.1) and ToString (ES5.1 9.8).

AddN	22	22	22	22	22	22	22	22	22	23	23	23	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 + Arg3 (Numeric addition, skips number check)
------	----	----	----	----	----	----	----	----	----	----	----	----	--

AsyncBreakCheck					91	91	91	91	94	94	96	(total size 0) Fast check for an async interrupt request.
-----------------	--	--	--	--	----	----	----	----	----	----	----	--

BitAnd	33	33	33	33	33	33	33	33	33	34	34	34	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 & Arg3 (JS bitwise AND)
--------	----	----	----	----	----	----	----	----	----	----	----	----	--

BitNot	11	11	11	11	11	11	11	11	11	12	12	12	Reg8, Reg8 (total size 2) Arg1 = ~Arg2 (Bitwise not)
--------	----	----	----	----	----	----	----	----	----	----	----	----	---

BitOr	35	35	35	35	35	35	35	35	35	36	36	36	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 Arg3 (JS bitwise OR)
-------	----	----	----	----	----	----	----	----	----	----	----	----	---

BitXor	34	34	34	34	34	34	34	34	34	35	35	35	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 ^ Arg3 (JS bitwise XOR)
--------	----	----	----	----	----	----	----	----	----	----	----	----	--

Call													Reg8, Reg8, UInt8 (total size 3)
	74	74	74	74	74	74	74	74	74	75	75	77	Call a function. Arg1 is the destination of the return value. Arg2 is the closure to invoke. Arg3 is the number of arguments, assumed to be found in reverse order from the end of the current frame.

													Reg8, Reg8, Reg8 (total size 3)
Call1	76	76	76	76	76	76	76	76	76	77	77	79	Call a function with one arg. Arg1 is the destination of the return value. Arg2 is the closure to invoke. Arg3 is the first argument.

Call2	78	78	78	78	78	78	78	78	78	79	79	81	Reg8, Reg8, Reg8, Reg8 (total size 4) Call a function with two args. Arg1 is the destination of the return value. Arg2 is the closure to invoke. Arg3 is the first argument. Arg4 is the second argument.
-------	----	----	----	----	----	----	----	----	----	----	----	----	--

Call3	79	79	79	79	79	79	79	79	79	80	80	82	Reg8, Reg8, Reg8, Reg8, Reg8 (total size 5) Call a function with three args. Arg1 is the destination of the return value. Arg2 is the closure to invoke. Arg3 is the first argument. Arg4 is the second argument. Arg5 is the third argument.
-------	----	----	----	----	----	----	----	----	----	----	----	----	--

Call4	80	80	80	80	80	80	80	80	80	81	81	83	Reg8, Reg8, Reg8, Reg8, Reg8, Reg8 (total size 6) Call a function with four args. Arg1 is the destination of the return value. Arg2 is the closure to invoke. Arg3 is the first argument. Arg4 is the second argument. Arg5 is the third argument. Arg6 is the fourth argument.
-------	----	----	----	----	----	----	----	----	----	----	----	----	---

													Reg8, UInt8, UInt8 (total size 3)
													Call a builtin function. Note this is NOT marked as a Ret target, because the callee is native and therefore never JS. Arg1 is the destination of the return value. Arg2 is the builtin number. Arg3 is the number of arguments, assumed to be found in reverse order from the end of the current frame. thisArg is set to "undefined".
CallBuiltin	84	84	84	84	84	84	84	84	84	85	85	87	

CallBuiltinLong									86	86	88	Reg8, UInt8, UInt32 (total size 6) Call a builtin function. Note this is NOT marked as a Ret target, because the callee is native and therefore never JS. Arg1 is the destination of the return value. Arg2 is the builtin number. Arg3 is the number of arguments, assumed to be found in reverse order from the end of the current frame. thisArg is set to "undefined".
-----------------	--	--	--	--	--	--	--	--	----	----	----	--

													Reg8, UInt8, UInt16 (function_id) (total size 4)
													Call a function directly without a closure. Arg1 is the destination of the return value. Arg2 is the number of arguments, assumed to be found in reverse order from the end of the current frame. The first argument 'this' is assumed to be created with CreateThis. Arg3 is index in the function table. Note that we expect the variable- sized argument to be last.
CallDirect	77	77	77	77	77	77	77	77	77	78	78	80	

CallDirectLongIndex	83	83	83	83	83	83	83	83	83	84	84	86	Reg8, UInt8, UInt32 (total size 6) Identical to CallDirect, but the function index is 32-bit.
---------------------	----	----	----	----	----	----	----	----	----	----	----	----	---

CallLong	81	81	81	81	81	81	81	81	81	82	82	84	Reg8, Reg8, UInt32 (total size 6) Identical to Call, but allowing more arguments.
----------	----	----	----	----	----	----	----	----	----	----	----	----	--

													Reg8 (total size 1)
Catch	86	86	86	86	86	86	86	86	86	89	89	91	Catch an exception (the first instruction in an exception handler). } catch(Arg1) {

													Reg8, Reg8 (total size 2)
CoerceThisNS	109	111	112	112	112	112	112	112	112	117	117	121	Coerce a value assumed to contain 'this' to an object using non-strict mode rules. Primitives are boxed, \c null or \c undefined produce the global object. Arg1 = coerce_to_object(Arg2)

CompleteGenerator												(total size 0)
	123	124	124	124	124	124	124	129	129	134	Set the generator status to complete, but do not return.	

Construct	75	75	75	75	75	75	75	75	75	76	76	78	Reg8, Reg8, UInt8 (total size 3) Call a constructor, with semantics identical to Call. Arg1 is the destination of the return value. Arg2 is the closure to invoke. Arg3 is the number of arguments, assumed to be found in reverse order from the end of the current frame. The first argument 'this' is assumed to be created with CreateThis.
-----------	----	----	----	----	----	----	----	----	----	----	----	----	---

ConstructLong	82	82	82	82	82	82	82	82	82	83	83	85	Reg8, Reg8, UInt32 (total size 6) Identical to Construct, but allowing more arguments.
---------------	----	----	----	----	----	----	----	----	----	----	----	----	---

CreateAsyncClosureLongIndex													Reg8, Reg8, UInt32 (function_id) (total size 6) Create a closure for an AsyncFunction. Arg1 is the register in which to store the closure. Arg2 is the current environment as loaded by GetEnvironment 0. Arg3 is index in the function table.
-----------------------------	--	--	--	--	--	--	--	--	--	--	--	--	---

													Reg8, Reg8, UInt16 (function_id) (total size 4)
CreateClosure	93	93	94	94	94	94	94	94	94	96	96	98	Create a closure. Arg1 is the register in which to store the closure. Arg2 is the current environment as loaded by GetEnvironment 0. Arg3 is index in the function table.

													Reg8, Reg8, UInt32 (function_id) (total size 6)
CreateClosureLongIndex	94	94	95	95	95	95	95	95	95	97	97	99	Create a closure. Arg1 is the register in which to store the closure. Arg2 is the current environment as loaded by GetEnvironment 0. Arg3 is index in the function table.

CreateEnvironment	47	47	47	47	47	47	47	47	47	48	48	50	Reg8 (total size 1) Create a new environment, to store values captured by closures.
-------------------	----	----	----	----	----	----	----	----	----	----	----	----	--

												Reg8, Reg8, UInt16 (function_id) (total size 4)
CreateGenerator		124	125	125	125	125	125	125	130	130	135	Create a generator. Arg1 is the register in which to store the generator. Arg2 is the current environment as loaded by GetEnvironment 0. Arg3 is index in the function table.

												Reg8, Reg8, UInt16 (function_id) (total size 4)
CreateGeneratorClosure		95	96	96	96	96	96	96	96	98	98	100 Create a closure for a GeneratorFunction. Arg1 is the register in which to store the closure. Arg2 is the current environment as loaded by GetEnvironment 0. Arg3 is index in the function table.

												Reg8, Reg8, UInt32 (function_id) (total size 6)
CreateGeneratorClosureLongIndex		96	97	97	97	97	97	97	99	99	101	Create a closure for a GeneratorFunction. Arg1 is the register in which to store the closure. Arg2 is the current environment as loaded by GetEnvironment 0. Arg3 is index in the function table.

												Reg8, Reg8, UInt32 (function_id) (total size 6)
CreateGeneratorLongIndex		125	126	126	126	126	126	126	131	131	136	Create a generator. Arg1 is the register in which to store the generator. Arg2 is the current environment as loaded by GetEnvironment 0. Arg3 is index in the function table.

CreateRegExp	117	119	120	120	120	120	120	120	125	125	130	Reg8, UInt32 (string_id), UInt32 (string_id), UInt32 (total size 13) Create a regular expression. Arg1 is the result. Arg2 is the string index of the pattern. Arg3 is the string index of the flags. Arg4 is the regexp bytecode index in the regexp table.
--------------	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	---

													Reg8, Reg8, Reg8 (total size 3)
													Allocate an empty, uninitialized object (immediately before a constructor).
CreateThis	95	97	98	98	98	98	98	98	98	102	102	104	Arg1 is the destination register. Arg2 is the prototype to assign it. Arg3 is the constructor closure that will be used*. * To allow internal constructors to have special objects allocated.

Debugger													(total size 0)
	89	89	90	90	90	90	90	90	90	93	93	95	Implementation dependent debugger action.

DebuggerCheckBreak	(total size 0)									
	90	90	91	91						Fast check for a debugger break request.

Dec												38	Reg8, Reg8 (total size 2) Arg1 = Arg2 - 1 (JS decrement, skips number check)
-----	--	--	--	--	--	--	--	--	--	--	--	----	--

													UInt32 (string_id) (total size 4)
DeclareGlobalVar	48	48	48	48	48	48	48	48	48	49	49	51	Declare a global variable by string table index. The variable will be set to undefined.

												Reg8, Reg8, UInt16 (string_id) (total size 4)
DelById	66	66	66	66	66	66	66	66	67	67	69	Delete a property by string table index. Arg1 = delete Arg2[stringtable[Arg3]]

													Reg8, Reg8, UInt32 (string_id) (total size 6)
DelByIdLong	67	67	67	67	67	67	67	67	67	68	68	70	Delete a property by string table index. Arg1 = delete Arg2[stringtable[Arg3]]

													Reg8, Reg8, Reg8 (total size 3)
DelByVal	70	70	70	70	70	70	70	70	70	71	71	73	Delete a property by value (when the value is not known at compile time). Arg1 = delete Arg2[Arg3]

													Reg8, Reg8 (total size 2)
DirectEval	87	87	87	87	87	87	87	87	87	90	90	92	ES6 18.2.1.1 PerformEval(Arg2, evalRealm, strictCaller=true, direct=true) Arg1 is the destination of the return value. Arg2 is the value to eval.

Div	25	25	25	25	25	25	25	25	25	26	26	26	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 / Arg3 (JS division)
-----	----	----	----	----	----	----	----	----	----	----	----	----	---

DivN	26	26	26	26	26	26	26	26	26	27	27	27	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 / Arg3 (Numeric division, skips number check)
------	----	----	----	----	----	----	----	----	----	----	----	----	---

Divu32										189	194	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 / Arg3 (32-bit unsigned integer division)
--------	--	--	--	--	--	--	--	--	--	-----	-----	--

Eq	13	13	13	13	13	13	13	13	13	14	14	14	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 == Arg3 (JS equality)
----	----	----	----	----	----	----	----	----	----	----	----	----	--

													Reg8, Reg8 (total size 2)
GetArgumentsLength	115	117	118	118	118	118	118	118	118	123	123	128	Get the length of the 'arguments' array. Arg1 is the result. Arg2 is the lazy loaded register. Arg1 = arguments.length

													Reg8, Reg8, Reg8 (total size 3)
GetArgumentsPropByVal	114	116	117	117	117	117	117	117	117	122	122	127	Get a property of the 'arguments' array by value. Arg1 is the result. Arg2 is the index. Arg3 is the lazy loaded register. Arg1 = arguments[Arg2]

[illegible]

													Reg8, Reg8, UInt8, UInt16 (string_id) (total size 5)
GetById	50	50	50	50	50	50	50	50	50	51	51	53	Get an object property by string table index. Arg1 = Arg2[stringtable[Arg4]] Arg3 is a cache index used to speed up the above operation.

													Reg8, Reg8, UInt8, UInt32 (string_id) (total size 7)
GetByIdLong	51	51	51	51	51	51	51	51	51	52	52	54	Get an object property by string table index. Arg1 = Arg2[stringtable[Arg4]] Arg3 is a cache index used to speed up the above operation.

													Reg8, Reg8, UInt8, UInt8 (string_id) (total size 4)
GetByIdShort	49	49	49	49	49	49	49	49	49	50	50	52	Get an object property by string table index. Arg1 = Arg2[stringtable[Arg4]] Arg3 is a cache index used to speed up the above operation.

													Reg8, Reg8, Reg8 (total size 3)
GetByVal	68	68	68	68	68	68	68	68	68	69	69	71	Get a property by value. Constants string values should instead use GetById. Arg1 = Arg2[Arg3]

													Reg8, UInt8 (total size 2)
GetEnvironment	38	38	38	38	38	38	38	38	38	39	39	41	Get an environment (scope) from N levels up the stack. 0 is the current environment, 1 is the caller's environment, etc.

GetGlobalObject	45	45	45	45	45	45	45	45	45	46	46	48	Reg8 (total size 1)
													Get the global object (the object in which global variables are stored).

												Reg8 (total size 1)
GetNewTarget	46	46	46	46	46	46	46	46	47	47	49	Obtain the value of NewTarget from the frame. Arg1 = NewTarget

													Reg8, Reg8, Reg8, Reg8, Reg8 (total size 5)
													Get the next property in the for..in iterator. Returns Arg1, which is the next property. Undefined if unavailable. Arg2 is the register that holds array of properties. Arg3 is the register that holds the object. Arg4 is the register that holds the iterating index. Arg5 is the register that holds the size of the property list.
GetNextPName	73	73	73	73	73	73	73	73	73	74	74	76	

													Reg8, Reg8, Reg8, Reg8 (total size 4)
GetPNameList	72	72	72	72	72	72	72	72	72	73	73	75	Get the list of properties from an object to implement for..in loop. Returns Arg1, which is the register that holds array of properties. Returns Undefined if the object is null/undefined. Arg2 is the register that holds the object. Arg3 is the register that holds the iterating index. Arg4 is the register that holds the size of the property list.

Greater	19	19	19	19	19	19	19	19	19	20	20	20	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 > Arg3 (JS greater-than)
---------	----	----	----	----	----	----	----	----	----	----	----	----	---

GreaterEq	20	20	20	20	20	20	20	20	20	21	21	21	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 >= Arg3 (JS greater-than-or-equals)
-----------	----	----	----	----	----	----	----	----	----	----	----	----	--

Inc												37	Reg8, Reg8 (total size 2) Arg1 = Arg2 + 1 (JS increment, skips number check)
-----	--	--	--	--	--	--	--	--	--	--	--	----	--

InstanceOf													Reg8, Reg8, Reg8 (total size 3)
	36	36	36	36	36	36	36	36	37	37	39		Check whether Arg2 contains Arg3 in its prototype chain. Note that this is not the same as JS instanceof. Pseudocode: Arg1 = prototypechain(Arg2).contains(Arg3)

IsIn	37	37	37	37	37	37	37	37	37	38	38	40	Reg8, Reg8, Reg8 (total size 3)
													Arg1 = Arg2 in Arg3 (JS relational 'in')

IteratorBegin											Reg8, Reg8 (total size 2)
											Arg1 [out] is the result iterator or index. Arg2 [in/out] is the source. Output for either the source or next method. If source is an array with an unmodified [Symbol.iterator], the result is 0. Else the result is source[Symbol.iterator] and the output is the .next() method on the iterator. See IR.md for IteratorBeginInst.

IteratorClose								129	129	134	134	139	Reg8, UInt8 (total size 2) Arg1 [in] is the iterator or array index. Arg2 is a bool indicating whether to ignore the inner exception. If the iterator is an object, call iterator.return(). If Arg2 is true, ignore exceptions which are thrown by iterator.return(). See IR.md for IteratorCloseInst.
---------------	--	--	--	--	--	--	--	-----	-----	-----	-----	-----	--

IteratorNext											Reg8, Reg8, Reg8 (total size 3)
						128	128	133	133	138	Arg1 [out] is the result, or undefined if done. Arg2 [in/out] is the iterator or index. Arg2 [in] is the source or the next method. If iterator is undefined, result = undefined. If iterator is a number: If iterator is less than source.length, return source[iterator++] Else iterator = undefined and result = undefined Else: n = iterator.next() If n.done, iterator = undefined and result = undefined. Else result = n.value See IR.md for IteratorNextInst.

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JEqual	159	168	169	169	169	169	172	172	177	177	182		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JEqualLong	160	169	170	170	170	170	173	173	178	178	183		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JGreater	143	152	153	153	153	153	156	156	161	161	166		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JGreaterEqual	151	160	161	161	161	161	164	164	169	169	174		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JGreaterEqualLong	152	161	162	162	162	162	165	165	170	170	175		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

												Addr8, Reg8, Reg8 (total size 3)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs,
JGreaterEqualN	155	164	165	165	165	165	168	168	173	173	178	we need to be able to say "jump when not less than to BB2" instead of
												"jump when less than to BB1".
												Since NaN comparisons always return false, "not less" != "greater or equal"

												Addr32, Reg8, Reg8 (total size 6)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs,
JGreaterEqualNLong	156	165	166	166	166	166	169	169	174	174	179	we need to be able to say "jump when not less than to BB2" instead of
												"jump when less than to BB1".
												Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JGreaterLong	144	153	154	154	154	154	157	157	162	162	167		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JGreaterN	147	156	157	157	157	157	160	160	165	165	170		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JGreaterNLong	148	157	158	158	158	158	161	161	166	166	171		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLess	127	136	137	137	137	137	140	140	145	145	150		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLessEqual	135	144	145	145	145	145	148	148	153	153	158		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLessEqualLong	136	145	146	146	146	146	149	149	154	154	159		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLessEqualN	139	148	149	149	149	149	152	152	157	157	162		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLessEqualNLong	140	149	150	150	150	150	153	153	158	158	163		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLessLong	128	137	138	138	138	138	141	141	146	146	151		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLessN	131	140	141	141	141	141	144	144	149	149	154		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JLessNLong	132	141	142	142	142	142	145	145	150	150	155		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JNotEqual	161	170	171	171	171	171	174	174	179	179	184		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotEqualLong	162	171	172	172	172	172	175	175	180	180	185		

												Addr8, Reg8, Reg8 (total size 3)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs,
JNotGreater	145	154	155	155	155	155	158	158	163	163	168	we need to be able to say "jump when not less than to BB2" instead of
												"jump when less than to BB1".
												Since NaN comparisons always return false, "not less" != "greater or equal"

												Addr8, Reg8, Reg8 (total size 3)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs,
JNotGreaterEqual	153	162	163	163	163	163	166	166	171	171	176	we need to be able to say "jump when not less than to BB2" instead of
												"jump when less than to BB1".
												Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotGreaterEqualLong	154	163	164	164	164	164	167	167	172	172	177		

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotGreaterEqualN	157	166	167	167	167	167	170	170	175	175	180		

												Addr32, Reg8, Reg8 (total size 6)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotGreaterEqualNLong	158	167	168	168	168	168	171	171	176	176	181	

												Addr32, Reg8, Reg8 (total size 6)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs,
JNotGreaterLong	146	155	156	156	156	156	159	159	164	164	169	we need to be able to say "jump when not less than to BB2" instead of
												"jump when less than to BB1".
												Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotGreaterN	149	158	159	159	159	159	162	162	167	167	172		

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JNotGreaterNLong	150	159	160	160	160	160	163	163	168	168	173		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotLess	129	138	139	139	139	139	142	142	147	147	152		

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JNotLessEqual	137	146	147	147	147	147	150	150	155	155	160		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

												Addr32, Reg8, Reg8 (total size 6)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotLessEqualLong	138	147	148	148	148	148	151	151	156	156	161	

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JNotLessEqualN	141	150	151	151	151	151	154	154	159	159	164		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

												Addr32, Reg8, Reg8 (total size 6)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs,
JNotLessEqualNLong	142	151	152	152	152	152	155	155	160	160	165	we need to be able to say "jump when not less than to BB2" instead of
												"jump when less than to BB1".
												Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotLessLong	130	139	140	140	140	140	143	143	148	148	153		

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JNotLessN	133	142	143	143	143	143	146	146	151	151	156		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JNotLessNLong	134	143	144	144	144	144	147	147	152	152	157		

													Addr8, Reg8, Reg8 (total size 3)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JStrictEqual	163	172	173	173	173	173	176	176	181	181	186		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

													Addr32, Reg8, Reg8 (total size 6)
													Not conditionals are required for NaN comparisons
													Since we want to be able to reorder targets to allow for fall-throughs,
JStrictEqualLong	164	173	174	174	174	174	177	177	182	182	187		we need to be able to say "jump when not less than to BB2" instead of
													"jump when less than to BB1".
													Since NaN comparisons always return false, "not less" != "greater or equal"

												Addr8, Reg8, Reg8 (total size 3)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs,
JStrictNotEqual	165	174	175	175	175	175	178	178	183	183	188	we need to be able to say "jump when not less than to BB2" instead of
												"jump when less than to BB1".
												Since NaN comparisons always return false, "not less" != "greater or equal"

												Addr32, Reg8, Reg8 (total size 6)
												Not conditionals are required for NaN comparisons
												Since we want to be able to reorder targets to allow for fall-throughs, we need to be able to say "jump when not less than to BB2" instead of "jump when less than to BB1". Since NaN comparisons always return false, "not less" != "greater or equal"
JStrictNotEqualLong	166	175	176	176	176	176	179	179	184	184	189	

Jmp													Addr8 (total size 1)
	119	126	127	127	127	127	130	130	135	135	140		Unconditional branch to Arg1.

JmpFalse													Addr8, Reg8 (total size 2)
	123	130	131	131	131	131	134	134	139	139	144		Conditional branches to Arg1 based on Arg2.

JmpFalseLong													Addr32, Reg8 (total size 5)
	124	131	132	132	132	132	135	135	140	140	145		Conditional branches to Arg1 based on Arg2.

JmpLong	120	127	128	128	128	128	131	131	136	136	141	Addr32 (total size 4)
												Unconditional branch to Arg1.

JmpTrue													Addr8, Reg8 (total size 2)
	121	128	129	129	129	129	132	132	137	137	142		Conditional branches to Arg1 based on Arg2.

JmpTrueLong													Addr32, Reg8 (total size 5)
	122	129	130	130	130	130	133	133	138	138	143		Conditional branches to Arg1 based on Arg2.

JmpUndefined													Addr8, Reg8 (total size 2)
	125	132	133	133	133	133	136	136	141	141	146		Jump if the value is undefined.

JmpUndefinedLong													Addr32, Reg8 (total size 5)
	126	133	134	134	134	134	137	137	142	142	147		Jump if the value is undefined.

LShift	30	30	30	30	30	30	30	30	30	31	31	31	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 << Arg3 (JS bitshift left)
--------	----	----	----	----	----	----	----	----	----	----	----	----	---

Less	17	17	17	17	17	17	17	17	17	18	18	18	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 < Arg3 (JS less-than)
------	----	----	----	----	----	----	----	----	----	----	----	----	--

													Reg8, Reg8, Reg8 (total size 3)
LessEq	18	18	18	18	18	18	18	18	18	19	19	19	Arg1 = Arg2 <= Arg3 (JS less-than-or-equals)

LoadConstDouble	101	103	104	104	104	104	104	104	108	108	110	Reg8, Double (total size 9) Load a constant double value.
-----------------	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--

LoadConstEmpty									111	111	115	Reg8 (total size 1) Load common constants.
----------------	--	--	--	--	--	--	--	--	-----	-----	-----	---

LoadConstFalse	107	109	110	110	110	110	110	110	115	115	119	Reg8 (total size 1)
												Load common constants.

LoadConstInt	100	102	103	103	103	103	103	103	103	107	107	109	Reg8, Imm32 (total size 5) Load a constant integer value.
--------------	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--

LoadConstNull	105	107	108	108	108	108	108	108	113	113	117	Reg8 (total size 1)
												Load common constants.

													Reg8, UInt16 (string_id) (total size 3)
LoadConstString	102	104	105	105	105	105	105	105	105	109	109	113	Load a constant string value by string table index.

													Reg8, UInt32 (string_id) (total size 5)
LoadConstStringLongIndex	103	105	106	106	106	106	106	106	110	110	114		Load a constant string value by string table index.

LoadConstTrue	106	108	109	109	109	109	109	109	109	114	114	118	Reg8 (total size 1)
													Load common constants.

LoadConstUInt8	99	101	102	102	102	102	102	102	102	106	106	108	Reg8, UInt8 (total size 2) Load a constant integer value.
----------------	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--

LoadConstUndefined													Reg8 (total size 1)
	104	106	107	107	107	107	107	107	112	112	116		Load common constants.

LoadConstZero	108	110	111	111	111	111	111	111	116	116	120	Reg8 (total size 1)
												Load common constants.

													Reg8, Reg8, UInt8 (total size 3)
LoadFromEnvironment	43	43	43	43	43	43	43	43	43	44	44	46	Load a value from an environment. Arg1 is the destination. Arg2 is the environment (as fetched by GetEnvironment). Arg3 is the environment index slot number.

													Reg8, Reg8, UInt16 (total size 4)
LoadFromEnvironmentL	44	44	44	44	44	44	44	44	44	45	45	47	Load a value from an environment. Arg1 is the destination. Arg2 is the environment (as fetched by GetEnvironment). Arg3 is the environment index slot number.

													Reg8, UInt8 (total size 2)
LoadParam	97	99	100	100	100	100	100	100	100	104	104	106	Load a function parameter by index. Starts at 0 with 'this'. Arg1 = Arg2 == 0 ? this : arguments[Arg2 - 1];

LoadParamLong	98	100	101	101	101	101	101	101	101	105	105	107	Reg8, UInt32 (total size 5) Like LoadParam, but allows accessing arguments >= 255.
---------------	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--

													Reg8 (total size 1)
LoadThisNS	110	112	113	113	113	113	113	113	113	118	118	122	Obtain the raw \c this value and coerce it to an object. Equivalent to: \code LoadParam Arg1, #0 CoerceThisNS Arg1, Arg1 \endcode

Loadi16											192	197	Reg8, Reg8, Reg8 (total size 3) Arg1 = HEAP16[Arg3 >> 1] (load signed 16-bit integer)
---------	--	--	--	--	--	--	--	--	--	--	-----	-----	---

Loadi32											194	199	Reg8, Reg8, Reg8 (total size 3) Arg1 = HEAP32[Arg3 >> 2] (load signed 32-bit integer)
---------	--	--	--	--	--	--	--	--	--	--	-----	-----	---

Loadi8											190	195	Reg8, Reg8, Reg8 (total size 3) Arg1 = HEAP8[Arg3] (load signed 8-bit integer)
--------	--	--	--	--	--	--	--	--	--	--	-----	-----	---

Loadu16	193	198	Reg8, Reg8, Reg8 (total size 3) Arg1 = HEAPU16[Arg3 >> 1] (load unsigned 16-bit integer)
---------	-----	-----	---

Loadu32											195	200	Reg8, Reg8, Reg8 (total size 3) Arg1 = HEAPU32[Arg3 >> 2] (load unsigned 32-bit integer)
---------	--	--	--	--	--	--	--	--	--	--	-----	-----	--

Loadu8	191	196	Reg8, Reg8, Reg8 (total size 3) Arg1 = HEAPU8[Arg3] (load unsigned 8-bit integer)
--------	-----	-----	--

Mod	27	27	27	27	27	27	27	27	27	28	28	28	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 % Arg3 (JS remainder)
-----	----	----	----	----	----	----	----	----	----	----	----	----	--

Mov	7	7	7	7	7	7	7	7	8	8	8	Reg8, Reg8 (total size 2)
	Arg1 = Arg2 (Register copy)											

MovLong	8	8	8	8	8	8	8	8	9	9	9	Reg32, Reg32 (total size 8)
												Arg1 = Arg2 (Register copy, long index)

Mul	23	23	23	23	23	23	23	23	23	24	24	24	Reg8, Reg8, Reg8 (total size 3)
													Arg1 = Arg2 * Arg3 (JS multiplication)

Mul32										187	192	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 * Arg3 (32-bit integer multiplication)
-------	--	--	--	--	--	--	--	--	--	-----	-----	---

MulN	24	24	24	24	24	24	24	24	24	25	25	25	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 * Arg3 (Numeric multiplication, skips number check)
------	----	----	----	----	----	----	----	----	----	----	----	----	--

Negate	9	9	9	9	9	9	9	9	10	10	10	Reg8, Reg8 (total size 2)
												Arg1 = -Arg2 (Unary minus)

Neq	15	15	15	15	15	15	15	15	15	16	16	16	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 != Arg3 (JS inequality)
-----	----	----	----	----	----	----	----	----	----	----	----	----	--

NewArray	6	6	6	6	6	6	6	6	7	7	7	Reg8, UInt16 (total size 3)
												Create a new array of a given size. Arg1 = new Array(Arg2)

													Reg8, UInt16, UInt16, UInt16 (total size 7)
ToArrayWithBuffer	4	4	4	4	4	4	4	4	5	5	5		Create an array from a static list of values, as for var=[1,2,3]. Any non-constant elements can be set afterwards with PutOwnByIndex. Arg1 is the destination. Arg2 is a preallocation size hint. Arg3 is the number of static elements. Arg4 is the index in the array buffer table.

												Reg8, UInt16, UInt16, UInt32 (total size 9)
ToArrayWithBufferLong	5	5	5	5	5	5	5	5	6	6	6	Create an array from a static list of values, as for var=[1,2,3]. Any non-constant elements can be set afterwards with PutOwnByIndex. Arg1 is the destination. Arg2 is a preallocation size hint. Arg3 is the number of static elements. Arg4 is the index in the array buffer table.

												Reg8 (total size 1)
NewObject	2	2	2	2	2	2	2	2	3	3	3	Create a new, empty Object using the built-in constructor (regardless of whether it was overridden). Arg1 = {}

													Reg8, UInt16, UInt16, UInt16, UInt16 (total size 9)
													Create an object from a static map of values, as for var={'a': 3}. Any non-constant elements can be set afterwards with PutOwnByInd.
NewObjectWithBuffer	0	0	0	0	0	0	0	0	0	1	1	1	Arg1 is the destination. Arg2 is a preallocation size hint. Arg3 is the number of static elements. Arg4 is the index in the object key buffer table. Arg5 is the index in the object val buffer table.

												Reg8, UInt16, UInt16, UInt32, UInt32 (total size 13)
NewObjectWithBufferLong	1	1	1	1	1	1	1	1	2	2	2	Create an object from a static map of values, as for var={'a': 3}. Any non-constant elements can be set afterwards with PutOwnByInd. Arg1 is the destination. Arg2 is a preallocation size hint. Arg3 is the number of static elements. Arg4 is the index in the object key buffer table. Arg5 is the index in the object val buffer table.

													Reg8, Reg8 (total size 2)
NewObjectWithParent	3	3	3	3	3	3	3	3	3	4	4	4	Create a new empty Object with the specified parent. If the parent is null, no parent is used. If the parent is not an object, the builtin Object.prototype is used. Otherwise the parent itself is used. Arg1 = the created object Arg2 = the parent.

Not	10	10	10	10	10	10	10	10	10	11	11	11	Reg8, Reg8 (total size 2) Arg1 = !Arg2 (Boolean not)
-----	----	----	----	----	----	----	----	----	----	----	----	----	---

													UInt16 (total size 2)
ProfilePoint	91	91	92	92	92	92	92	92	92	95	95	97	Define a profile point. Arg1 is the function local profile point index. The first one will have the largest index. If there are more than 2^16 profile points in the function, all the overflowed profile points have index zero.

PutById	54	54	54	54	54	54	54	54	54	55	55	57	Reg8, Reg8, UInt8, UInt16 (string_id) (total size 5) Set an object property by string index. Arg1[stringtable[Arg4]] = Arg2.
---------	----	----	----	----	----	----	----	----	----	----	----	----	--

PutByIdLong	55	55	55	55	55	55	55	55	55	56	56	58	Reg8, Reg8, UInt8, UInt32 (string_id) (total size 7) Set an object property by string index. Arg1[stringtable[Arg4]] = Arg2.
-------------	----	----	----	----	----	----	----	----	----	----	----	----	--

													Reg8, Reg8, Reg8 (total size 3)
PutByVal	69	69	69	69	69	69	69	69	69	70	70	72	Set a property by value. Constant string values should instead use GetById (unless they are array indices according to ES5.1 section 15.4, in which case this is still the right opcode). Arg1[Arg2] = Arg3

													Reg8, Reg8, UInt16 (string_id) (total size 4)
PutNewOwnById	59	59	59	59	59	59	59	59	59	60	60	62	Create a new own property on an object. This is similar to PutById, but the destination must be an object, it only deals with own properties, ignoring the prototype chain, and the property must not already be defined. Similarly to PutById, the property name cannot be a valid array index. Arg1 is the destination object, which is known to be an object. Arg2 is the value to write. Arg3 is the string table ID of the property name. Arg1[stringtable[Arg3]] = Arg2

													Reg8, Reg8, UInt32 (string_id) (total size 6)
PutNewOwnByIdLong	60	60	60	60	60	60	60	60	60	61	61	63	Create a new own property on an object. This is similar to PutById, but the destination must be an object, it only deals with own properties, ignoring the prototype chain, and the property must not already be defined. Similarly to PutById, the property name cannot be a valid array index. Arg1 is the destination object, which is known to be an object. Arg2 is the value to write. Arg3 is the string table ID of the property name. Arg1[stringtable[Arg3]] = Arg2

													Reg8, Reg8, UInt8 (string_id) (total size 3)
PutNewOwnByIdShort	58	58	58	58	58	58	58	58	58	59	59	61	Create a new own property on an object. This is similar to PutById, but the destination must be an object, it only deals with own properties, ignoring the prototype chain, and the property must not already be defined. Similarly to PutById, the property name cannot be a valid array index. Arg1 is the destination object, which is known to be an object. Arg2 is the value to write. Arg3 is the string table ID of the property name. Arg1[stringtable[Arg3]] = Arg2

													Reg8, Reg8, UInt16 (string_id) (total size 4)
													Create a new non-enumerable own property on an object. This is the same as
PutNewOwnNEById	61	61	61	61	61	61	61	61	61	62	62	64	PutNewOwnById, but creates the property with different enumerability. Arg1 is the destination object. Arg2 is the value to write. Arg3 is the string table ID of the property name. Arg1[stringtable[Arg3]] = Arg2

													Reg8, Reg8, UInt32 (string_id) (total size 6)
PutNewOwnNEByIdLong	62	62	62	62	62	62	62	62	62	63	63	65	Create a new non-enumerable own property on an object. This is the same as PutNewOwnById, but creates the property with different enumerability. Arg1 is the destination object. Arg2 is the value to write. Arg3 is the string table ID of the property name. Arg1[stringtable[Arg3]] = Arg2

													Reg8, Reg8, UInt8 (total size 3)
PutOwnByIndex	63	63	63	63	63	63	63	63	63	64	64	66	Assign a value to a constant integer own property which will be created as enumerable. This is used (potentially in conjunction with NewArrayWithBuffer) for arr=[foo,bar] initializations. Arg1[Arg3] = Arg2;

													Reg8, Reg8, UInt32 (total size 6)
PutOwnByIndexL	64	64	64	64	64	64	64	64	64	65	65	67	Assign a value to a constant integer own property which will be created as enumerable. This is used (potentially in conjunction with NewArrayWithBuffer) for arr=[foo,bar] initializations. Arg1[Arg3] = Arg2;

													Reg8, Reg8, Reg8, UInt8 (total size 4)
													Set an own property identified by value.
													Arg1 is the destination object.
													Arg2 is the value to write.
													Arg3 is the property name.
													Arg4 : bool -> enumerable. If true, the property is created as enumerable,
													non-enumerable otherwise.
													Arg1[Arg3] = Arg2;
PutOwnByVal	65	65	65	65	65	65	65	65	65	66	66	68	

													Reg8, Reg8, Reg8, Reg8, UInt8 (total size 5)
													Add a getter and a setter for a property by value. Object.defineProperty(Arg1, Arg2, { get: Arg3, set: Arg4 }). Arg1 is the target object which will have a property defined. Arg2 is the property name Arg3 is the getter closure or undefined Arg4 is the setter closure or undefined Arg5 : boolean - if true, the property will be enumerable.
PutOwnGetterSetterByVal	71	71	71	71	71	71	71	71	71	72	72	74	

RShift	31	31	31	31	31	31	31	31	31	32	32	32	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 >> Arg3 (JS signed bitshift right)
--------	----	----	----	----	----	----	----	----	----	----	----	----	--

													Reg8 (total size 1)
ReifyArguments	116	118	119	119	119	119	119	119	119	124	124	129	Create an actual 'arguments' array, if get-by-index and length isn't enough. Arg1 is the lazy loaded register, which afterwards will contain a proper object that can be used by non- *Arguments* opcodes like Return.

												Reg8, Reg8 (total size 2)
ResumeGenerator		122	123	123	123	123	123	123	128	128	133	Resume generator by performing one of the following user-requested actions: - next(val): Set Arg1 to val, Arg2 to false, run next instruction - return(val): Set Arg1 to val, Arg2 to true, run next instruction - throw(val): Throw val as an error Arg1 is the result provided by the user. Arg2 is a boolean which is true if the user requested a return().

													Reg8 (total size 1)
Ret	85	85	85	85	85	85	85	85	85	88	88	90	Return a value from the current function. return Arg1;

												Addr8 (total size 1)
SaveGenerator		134	135	135	135	135	138	138	143	143	148	Save the provided value, yield, and signal the VM to restart execution at the provided target.

												Addr32 (total size 4)
SaveGeneratorLong		135	136	136	136	136	139	139	144	144	149	Save the provided value, yield, and signal the VM to restart execution at the provided target.

													Reg8, Reg8, Reg8 (total size 3)
													Choose the result of a constructor: 'this' or a returned object. Arg1 is the result. Arg2 is the 'this' object used for the constructor. Arg3 is the constructor's return value. Arg1 = Arg3 instanceof Object ? Arg3 : Arg2
SelectObject	96	98	99	99	99	99	99	99	99	103	103	105	

												(total size 0)
StartGenerator		121	122	122	122	122	122	122	127	127	132	Start the generator by jumping to the next instruction to begin. Restore the stack frame if this generator has previously been suspended.

Store16											197	202	Reg8, Reg8, Reg8 (total size 3) HEAP16[Arg2] = Arg3 (store signed or unsigned 16-bit integer)
---------	--	--	--	--	--	--	--	--	--	--	-----	-----	---

Store32	198	203	Reg8, Reg8, Reg8 (total size 3)
			HEAP32[Arg2] = Arg3 (store signed or unsigned 32-bit integer)

Store8											196	201	Reg8, Reg8, Reg8 (total size 3) HEAP8[Arg2] = Arg3 (store signed or unsigned 8-bit integer)
--------	--	--	--	--	--	--	--	--	--	--	-----	-----	--

													Reg8, UInt8, Reg8 (total size 3)
StoreNPToEnvironment	41	41	41	41	41	41	41	41	41	42	42	44	Store a value in an environment. StoreNPToEnvironment[L] store a non-pointer value in an environment Arg1 is the environment (as fetched by GetEnvironment). Arg2 is the environment index slot number. Arg3 is the value.

													Reg8, UInt16, Reg8 (total size 4)
StoreNPToEnvironmentL	42	42	42	42	42	42	42	42	42	43	43	45	Store a value in an environment. StoreNPToEnvironment[L] store a non-pointer value in an environment Arg1 is the environment (as fetched by GetEnvironment). Arg2 is the environment index slot number. Arg3 is the value.

													Reg8, UInt8, Reg8 (total size 3)
StoreToEnvironment	39	39	39	39	39	39	39	39	39	40	40	42	Store a value in an environment. StoreNPToEnvironment[L] store a non-pointer value in an environment Arg1 is the environment (as fetched by GetEnvironment). Arg2 is the environment index slot number. Arg3 is the value.

													Reg8, UInt16, Reg8 (total size 4)
StoreToEnvironmentL	40	40	40	40	40	40	40	40	40	41	41	43	Store a value in an environment. StoreNPToEnvironment[L] store a non-pointer value in an environment Arg1 is the environment (as fetched by GetEnvironment). Arg2 is the environment index slot number. Arg3 is the value.

StrictEq	14	14	14	14	14	14	14	14	14	15	15	15	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 === Arg3 (JS strict equality)
----------	----	----	----	----	----	----	----	----	----	----	----	----	--

StrictNeq	16	16	16	16	16	16	16	16	16	17	17	17	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 !== Arg3 (JS strict inequality)
-----------	----	----	----	----	----	----	----	----	----	----	----	----	--

Sub	28	28	28	28	28	28	28	28	29	29	29	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 - Arg3 (JS subtraction)
-----	----	----	----	----	----	----	----	----	----	----	----	--

Sub32	186	191	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 - Arg3 (32-bit integer subtraction)
-------	-----	-----	--

SubN	29	29	29	29	29	29	29	29	29	30	30	30	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 - Arg3 (Numeric subtraction, skips number check)
------	----	----	----	----	----	----	----	----	----	----	----	----	---

													Reg8, UInt32, Addr32, UInt32, UInt32 (total size 17)
													Jump table switch - using a table of offset, jump to the offset of the given input or to the default block if out of range (or not right type) Arg 1 is the value to be branched upon Arg 2 is the relative offset of the jump table to be used by this instruction. Jump tables are appended to the bytecode. Arg 3 is the relative offset for the "default" jump. Arg 4 is the unsigned min value, if arg 1 is less than this value jmp to default block Arg 5 is the unsigned max value, if arg 1 is greater than this value jmp to default block. Given the above, the jump table entry for a given value (that is in range) is located at offset $ip + arg2 + arg1 - arg4$. We subtract $arg4$ to avoid wasting space when compiling denses switches that do not start at zero. Note that Arg2 is <i>*unaligned*</i> it is dynamically aligned at runtime.
SwitchImm	118	120	121	121	121	121	121	121	121	126	126	131	

Throw	88	88	88	88	88	88	88	88	88	91	91	93	Reg8 (total size 1) Throw an exception. throw Arg1;
-------	----	----	----	----	----	----	----	----	----	----	----	----	---

ThrowIfEmpty									92	92	94	Reg8, Reg8 (total size 2) If Arg2 is Empty, throw ReferenceError, otherwise move it into Arg1. Arg1 is the destination of the return value Arg2 is the value to check
--------------	--	--	--	--	--	--	--	--	----	----	----	---

ThrowIfUndefinedInst			89	89	89	89	89	89					Reg8 (total size 1) Throw ReferenceError if the operand is HermesValue::undefined
----------------------	--	--	----	----	----	----	----	----	--	--	--	--	---

													Reg8, Reg8 (total size 2)
ToInt32	112	114	115	115	115	115	115	115	120	120	125	Convert a value to a 32-bit signed integer. Arg1 = Arg2 0	

													Reg8, Reg8 (total size 2)
ToNumber	111	113	114	114	114	114	114	114	114	119	119	123	Convert a value to a number. Arg1 = Arg2 - 0

ToNumeric															Reg8, Reg8 (total size 2) 124 Convert a value to a numeric. Arg1 = ToNumeric(Arg2)
-----------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	---

													Reg8, Reg8, UInt8, UInt16 (string_id) (total size 5)
TryGetById	52	52	52	52	52	52	52	52	52	53	53	55	Get an object property by string table index, or throw if not found. This is similar to GetById, but intended for use with global variables where Arg2 = GetGlobalObject.

													Reg8, Reg8, UInt8, UInt32 (string_id) (total size 7)
TryGetByIdLong	53	53	53	53	53	53	53	53	53	54	54	56	Get an object property by string table index, or throw if not found. This is similar to GetById, but intended for use with global variables where Arg2 = GetGlobalObject.

													Reg8, Reg8, UInt8, UInt16 (string_id) (total size 5)
TryPutById	56	56	56	56	56	56	56	56	56	57	57	59	Set an object property by string index, or throw if undeclared. This is similar to PutById, but intended for use with global variables where Arg1 = GetGlobalObject.

													Reg8, Reg8, UInt8, UInt32 (string_id) (total size 7)
TryPutByIdLong	57	57	57	57	57	57	57	57	57	58	58	60	Set an object property by string index, or throw if undeclared. This is similar to PutById, but intended for use with global variables where Arg1 = GetGlobalObject.

TypeOf	12	12	12	12	12	12	12	12	13	13	13	Reg8, Reg8 (total size 2) Arg1 = typeof Arg2 (JS typeof)
--------	----	----	----	----	----	----	----	----	----	----	----	---

URshift	32	32	32	32	32	32	32	32	32	33	33	33	Reg8, Reg8, Reg8 (total size 3) Arg1 = Arg2 >>> Arg3 (JS unsigned bitshift right)
---------	----	----	----	----	----	----	----	----	----	----	----	----	---

												(total size 0)
Unreachable	92	92	93	93	93	93	93	93	0	0	0	Unreachable opcode for stubs and similar. This is first so that it has the value zero.